I-ILIA-202 : Advanced Deep Learning
Project report

---

# Transformers for Time Series Forecasting

---


***Students :***
Cyril TAQUET
Thibaut WINDAL

***Teacher :***
Saïd. MAHMOUDI
***Assistants :***
O. AMEL
A. COOLS


2025 - 2026

# Contents

# Abstract

Time series forecasting plays a crucial role in modeling and predicting energy consumption, where accurate forecasts are essential for planning and operational decision-making. In this paper, we study the problem of electrical energy consumption forecasting and compare four widely used approaches: ARIMA, Random Forest regression, Long Short-Term Memory (LSTM) networks, and Transformer-based models.

The models are evaluated on three forecasting tasks of increasing difficulty: one-step-ahead prediction, one-day-ahead forecasting, and long-term recursive forecasting over the entire test set. Performance is assessed using standard error metrics, including MAE, MASE, and SMAPE.

Our results show that Transformer models achieve the best performance for short-term, one-step-ahead forecasting, while ARIMA struggle in multi-step settings. For long-term recursive forecasting, LSTM models demonstrate superior stability and robustness, maintaining acceptable accuracy for prediction horizons of up to 15 days.


# Introduction

Time series forecasting is the process of analyzing historical observations collected over time in order to model underlying patterns and predict future values. A time series is a specific type of data composed of sequentially ordered observations, typically recorded at regular time intervals. Such data are widely used to study temporal evolution in a broad range of applications, including system monitoring, financial markets, and demand estimation.

In this work, we focus on the problem of forecasting electrical energy consumption. Accurate energy demand forecasting is a critical task for resource planning, load balancing, and operational decision-making. The temporal nature of consumption data, combined with recurring patterns and long-term trends, makes time series modeling particularly well suited for this application.

This report is structured as follows. First, we review related work and existing approaches from the state of the art. We then describe our methodology and experimental setup, including data preprocessing, model selection, and evaluation metrics. Finally, we present and analyze the experimental results before concluding with a summary of the main findings and perspectives for future work.

# 1 Related work

Time series forecasting has been an active area of research since at least the 1980s and has found applications in a wide range of domains, including energy consumption, financial markets, and healthcare. Due to its practical importance, the field has attracted substantial research attention over the past decades.

Since the 1980s, major advances in computing power and data availability have significantly influenced the evolution of time series forecasting methods. Early approaches were primarily based on signal processing and statistical modeling techniques.

Machine learning methods for time series forecasting have evolved from classical algorithms such as random forests to neural network-based models, including multilayer per-

ceptrons (MLP), convolutional neural networks (CNN), recurrent neural networks (RNN), and, more recently, transformer-based architectures. As a result, a wide variety of models are now available, each with distinct assumptions, strengths, and limitations [1, 2].

One of the earliest and most widely used statistical models for time series forecasting is the ARIMA model, which stands for *AutoRegressive Integrated Moving Average*. ARIMA models are rooted in classical time series analysis and are designed to capture temporal dependencies through three main components:

- Autoregressive (AR): models the dependence between an observation and a number of its past values.
- Integrated (I): applies differencing to the time series in order to achieve stationarity.
- Moving Average (MA): models the dependence between an observation and past forecast errors, which is particularly useful for capturing short-term fluctuations.

The parameters of an ARIMA model are traditionally identified using the Box–Jenkins methodology [5], although alternative approaches exist. Modern statistical software libraries provide automated procedures for parameter selection.

Despite its effectiveness, the ARIMA model has limitations, particularly when dealing with seasonal patterns. Several extensions have been proposed to address these issues, such as the Seasonal ARIMA (SARIMA) model [7], which explicitly incorporates seasonal components.

Among deep learning approaches, several model architectures can be applied to time series forecasting. Some families of models are particularly well suited for sequential data, including recurrent neural networks (RNNs) and transformer-based architectures.

The core idea behind the RNN family is to process sequential data by iteratively updating an internal hidden state. At each time step, the model receives an input value (or vector), which is used together with the previous hidden state to update the internal representation of the sequence. This hidden state serves as a form of memory and can be used to generate an output, such as a prediction of the next time step. Due to this recursive structure, RNNs are, in theory, capable of handling sequences of arbitrary length [4]. Figure 1 illustrates the general concept of an RNN.

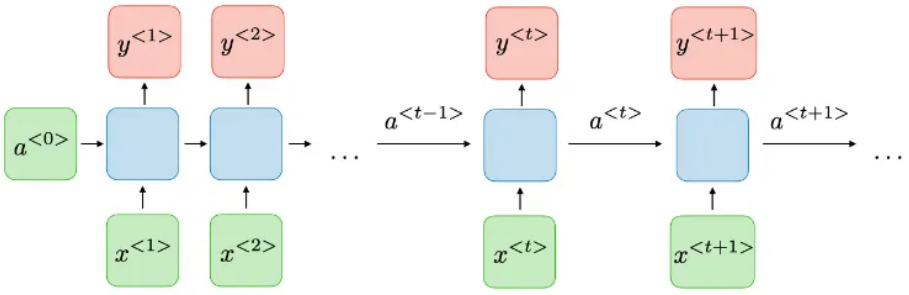


Figure 1: Recurrent Neural Network (RNN) concept where $x$ represent the input, $a$ the hidden state and $y$ the output value

Within the RNN family, Long Short-Term Memory (LSTM) networks represent an important extension. Several variants of LSTM architectures exist, all designed to address

key limitations of standard RNNs. In particular, LSTMs mitigate the vanishing gradient problem, which makes it difficult for vanilla RNNs to learn long-term dependencies in long sequences. By introducing gating mechanisms, LSTMs are better suited to modeling long-range temporal relationships. However, both RNNs and LSTMs remain constrained by the use of a single hidden-state representation, which can act as a bottleneck when modeling complex dependencies.[9]

In contrast, transformer-based models rely on the attention mechanism to overcome this limitation. Originally developed for natural language processing tasks [8], transformers do not rely on recurrent computations. Instead, they process sequences by allowing each element to directly attend to all other elements in the sequence. Since a sentence can be viewed as a time-ordered sequence of tokens, this framework naturally extends to time series forecasting and has been widely adopted in this context.

The attention mechanism, and more specifically self-attention, dynamically assigns importance weights to different elements of the input sequence. This enables the model to capture both short-term and long-term dependencies without relying on a single compressed memory representation, thereby alleviating the bottleneck inherent in RNN-based models.

Recent state-of-the-art transformer-based models for time series forecasting have demonstrated strong empirical performance and outperform traditional statistical and recurrent neural network approaches [10]. Additionally, they report improved data efficiency compared to earlier deep learning models, making these architectures particularly attractive in practical forecasting settings.

# 2 Proposed approach

Before considering specific forecasting models, an initial analysis of the dataset was conducted in order to understand its structure and determine which modeling approaches would be most appropriate.

The dataset used in this study is the *Electricity Load Diagrams* dataset [6], which contains electricity consumption data for 370 clients. For each client, measurements are recorded at 15-minute intervals from the beginning of 2011 to the end of 2014, resulting in up to 140,256 observations per client.

The dataset documentation indicates that some of the 370 clients were added after 2011, and that their consumption values prior to the acquisition date are set to zero. These artificially introduced zero values do not correspond to actual consumption and therefore need to be handled carefully during model selection.

The objective of this work is to construct a forecasting model for a single client. In order to maximize the amount of available training data and ensure consistency across experiments, we considered only clients whose data spans the entire observation period starting in 2011. This choice allows us to later reduce the training horizon if necessary, without having to change the selected client due to data scarcity.

Although the dataset is reported to contain no missing values, exploratory data analysis revealed that some clients exhibit extended periods of zero consumption following non-zero activity. Such abrupt and persistent drops in consumption are likely to reflect data collection issues or atypical behavior rather than realistic usage patterns. Consequently, these clients were excluded from further analysis.

Based on these criteria, we selected clients whose minimum recorded consumption is strictly greater than zero. This filtering step resulted in a subset of 90 clients. From this subset, the client **MT_292** was selected for further experimentation. Figure 2 illustrates the electricity consumption of this client over the full observation period.

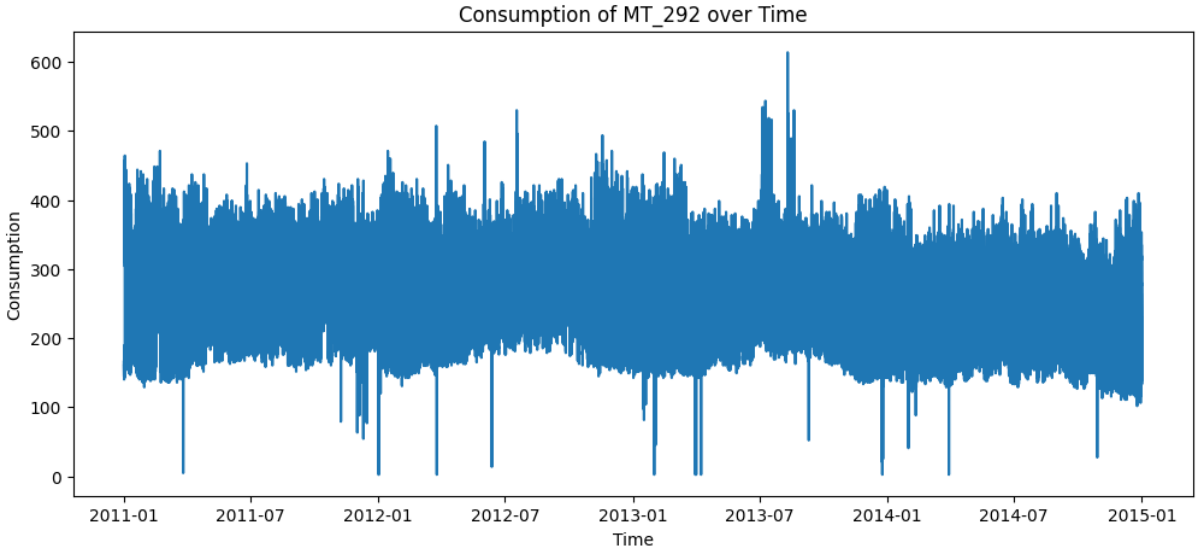


Figure 2: Electricity consumption of client **MT_292** from 2011 to 2014

For the selected client, the dataset contains a single primary feature: electricity consumption. In this study, we focus exclusively on this univariate time series. While additional features could be derived from the timestamp (e.g., day of the week or time of day), no explicit feature engineering based on calendar information is performed.

The objective of the proposed models is to forecast electricity consumption over a full day, corresponding to 96 consecutive time steps, using historical observations as input.

Four different forecasting approaches are considered. Among them, Long Short-Term Memory (LSTM) networks and transformer-based models are natural candidates, as they are well suited for sequential prediction tasks and have been successfully applied to time series forecasting in prior work.

Both architectures are evaluated in order to assess the trade-off between model complexity and predictive performance. While transformer models are particularly effective at capturing complex and long-range dependencies, such capabilities may not be strictly necessary for this task. Given the structure of the data, an LSTM model may achieve comparable performance while being computationally lighter and easier to use in practice.

In addition to deep learning approaches, we include classical statistical models as baselines. Since the data exhibits clear seasonal patterns, a Seasonal ARIMA (SARIMA) model would be a natural choice. However, due to the high computational cost and parameter complexity of SARIMA for long time series, we instead employ a standard ARIMA model. Preliminary experiments showed that SARIMA models can be trained using relatively short historical windows (e.g., one to two weeks of data), which is significantly smaller than the full dataset. A known limitation of ARIMA-based approaches is that they are designed for one-step-ahead forecasting; multi-step forecasts therefore require recursive predictions, which can lead to rapid error accumulation.

Finally, a Random Forest Regressor is considered. Although this model is not inherently designed for time series forecasting, it can be adapted to the task by using sliding windows of past observations as input features. This approach allows us to evaluate how well a non-sequential machine learning model performs when repurposed for forecasting tasks.

To enable a fair comparison between models, several evaluation metrics are used:

- **Mean Absolute Error (MAE)**:

$$\text{MAE} = \frac{1}{n}\sum_{i=1}^{n}|y_i - \hat{y}_i|,$$

  which directly measures the average magnitude of forecasting errors. Since MAE depends on the scale of the data, it is not suitable for comparing results across different clients.

- **Mean Absolute Scaled Error (MASE)**:

$$\text{MASE} = \frac{\text{MAE}}{\text{MAE}_{\text{naive}}},$$

  where $\text{MAE}_{\text{naive}}$ corresponds to the MAE of a naive forecasting model that predicts the last observed value used on the train set. Values below 1 indicate an improvement over the naive baseline.

- **Symmetric Mean Absolute Percentage Error (SMAPE)**:

$$\text{SMAPE} = \frac{2}{n}\sum_{i=1}^{n}\frac{|y_i - \hat{y}_i|}{|y_i| + |\hat{y}_i|},$$

  which expresses forecasting error as a relative percentage.

In all cases, $y$ denotes the true target value, $\hat{y}$ the predicted value, and $n$ the number of observations used to compute the metric. Lower values indicate better predictive performance.

Each model is evaluated on three forecasting tasks: one-step-ahead prediction, one-day-ahead forecasting, and long-horizon forecasting over the entire test set. The first two tasks assess short-term predictive performance, while the final task evaluates the models' ability to maintain accuracy over extended horizons.

We anticipate that long-term forecasts will exhibit degraded performance due to the accumulation of prediction errors over time. To further analyze this effect, additional evaluations are conducted at intermediate horizons (one day, one week, one month, and the full test period) in order to identify the point at which error accumulation renders forecasts unreliable.

# 3 Experimental setup

A key component of this work concerns how the dataset is structured and partitioned in order to train and evaluate the forecasting models in a consistent and reproducible manner.

Although recurrent and transformer-based models are capable of processing long sequences, using the full time series as a single training example is neither efficient nor effective. Instead, we adopt a sliding-window approach to construct the training dataset. Each sample consists of an input window and a corresponding forecasting window. The input window is provided to the model, which is tasked with predicting the values in the forecast window. Figure 3 illustrates this preprocessing procedure.

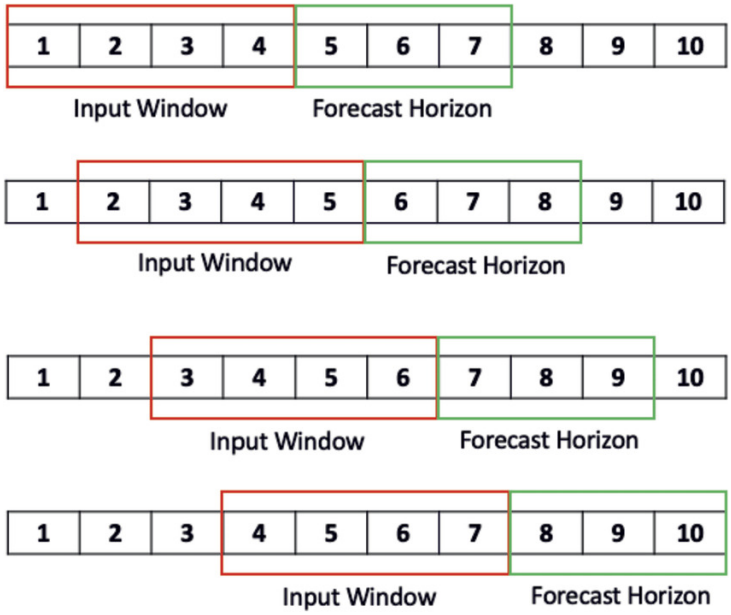


Figure 3: Sliding-window preprocessing applied to the time series

The input window length is set to 672 time steps, corresponding to one week of data, while the forecasting horizon is fixed to one day, i.e., 96 time steps. This configuration enables the generation of a large number of training samples from a single time series, thereby increasing data variability during training.

This design choice limits the model's ability to explicitly capture dependencies spanning more than one week. However, this constraint is considered acceptable in the present context, as long-term consumption trends (e.g., yearly variations) are likely driven by external factors specific to the client. In contrast, daily and weekly consumption patterns are relatively stable and therefore more amenable to short- and medium-term forecasting.

The same sliding-window representation is also used to train the Random Forest Regressor, which requires fixed-size input and output vectors. Without this transformation, applying such a model to time series forecasting would not be feasible.

After window generation, the data is split chronologically into three subsets: 80% for training, 10% for validation, and 10% for testing. The split is performed in temporal order without random shuffling, ensuring that no future information is leaked into the training process and that all experiments are fully reproducible.

The training set is used to fit model parameters, the validation set to tune hyperparameters, and the test set to evaluate final performance.

This data partitioning strategy is not directly applicable to ARIMA-based models, which require a contiguous time series as input. Nevertheless, we retain the same temporal split and evaluate ARIMA forecasts on the validation and test segments in a consistent manner.

For ARIMA modeling, we use the *pmdarima* library, which provides automated and widely used procedures for selecting optimal model parameters. The Random Forest Regressor is implemented using *scikit-learn*, which offers a well-established and efficient implementation with a limited number of tunable hyperparameters.

Recurrent neural networks and transformer-based models are implemented using the *tsai* library, which is built on top of *PyTorch*. WE use *TSTplus* model from *tsai*. Model parameters are optimized using the Adam optimizer, and training is performed using mini-batches to improve computational efficiency.

The L1 loss function is used for training neural network models, as it is more robust to outliers than the L2 loss. This choice is motivated by the presence of occasional high/low-consumption peaks in the dataset, as illustrated in Figure 2.

All code and experimental configurations are publicly available at `https://github.com/orbitalgamer/Advanced-Deep-Learning`.

# 4 Experimental results

The first experiment evaluates one-step-ahead forecasting performance. In order to avoid retraining separate models, each model is configured to predict the next 96 time steps, and only the first predicted value is retained for evaluation. The resulting performance metrics are reported in Table 1.

Table 1: Results of one-step-ahead forecasting

| Method | MAE | MASE | SMAPE |
|---|---|---|---|
| ARIMA | 18.18 | 1.04 | 0.078 |
| Random Forest | 19.41 | 1.11 | 0.087 |
| LSTM | 16.53 | 0.95 | 0.072 |
| Transformer | 14.55 | 0.83 | 0.063 |

ARIMA achieves relatively competitive error metrics in this setting. However, inspection of its forecasts (Figure 4) reveals that the model predicts values close to the last observed data point. This behavior is consistent with the limited autoregressive order used, which was constrained by the length of the training window. In addition, the small error magnitude can be partially explained by the high sampling frequency of the data, where consecutive observations tend to be similar, making naive or near-naive forecasts appear effective under one-step evaluation metrics.

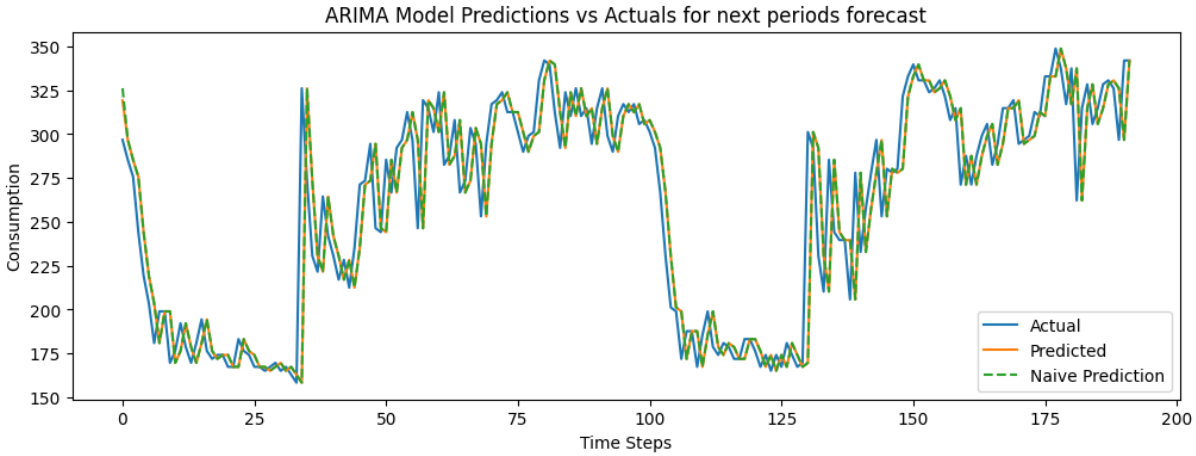


Figure 4: One-step-ahead forecasting using the ARIMA model (first 194 points)

Among the remaining models, the transformer achieves the best overall performance for one-step-ahead forecasting across all metrics. Notably, only the LSTM and transformer models obtain MASE values below 1, indicating performance superior to a naive persistence baseline.

The second experiment evaluates multi-step forecasting over a full day (96 time steps). The corresponding results are presented in Table 2.

Table 2: Results of one-day-ahead forecasting

| Method | MAE | MASE | SMAPE |
|---|---|---|---|
| ARIMA | 62.61 | 3.59 | 0.265 |
| Random Forest | 22.96 | 1.32 | 0.102 |
| LSTM | 23.21 | 1.33 | 0.102 |
| Transformer | 23.86 | 1.37 | 0.107 |

As expected, forecasting performance degrades for all models when predicting a full day ahead, reflecting the increased uncertainty associated with longer forecasting horizons. The ARIMA model performs particularly poorly in this setting, as errors accumulate rapidly during recursive multi-step prediction. While the initial forecasted steps are reasonable, the model quickly converges to constant predictions, resulting in large overall errors.

Among the remaining approaches, no single model clearly dominates across all metrics. However, the Random Forest Regressor exhibits the smallest relative degradation in performance when transitioning from one-step to one-day forecasting, suggesting greater robustness to increasing prediction horizons in this experimental setup.

It is important to note that the interpretation of the MASE metric becomes less straightforward in a multi-step forecasting setting. By definition, MASE is normalized using the error of a one-step-ahead naive forecasting model computed on the training set. The primary purpose of this metric is to assess whether a forecasting model outperforms a simple persistence baseline.

In the context of long-horizon forecasting, this normalization can be misleading. If a naive forecaster is applied to the test set while only observing the first value of the

forecasting horizon, it will repeatedly predict this value for all subsequent time steps. Such behavior would result in a substantially larger MAE over a 96-step horizon, and consequently yield much smaller MASE values when used as a normalization baseline.

As discussed previously, the ARIMA model produces forecasts that are close to a naive persistence strategy and achieves an MAE of 62.61 for one-day-ahead forecasting. Under this perspective, the effective MASE values for the remaining models would be expected to lie approximately in the range of 0.35 to 0.40, indicating significantly better performance relative to a naive multi-step baseline.

Long-term forecasting performance is illustrated in Figures 5, 6, and 7. These figures show a rapid increase in error metrics as the forecasting horizon grows, highlighting the accumulation of prediction errors inherent to recursive forecasting strategies. Since predicted values are reused as inputs for subsequent predictions, even small inaccuracies can compound over time.

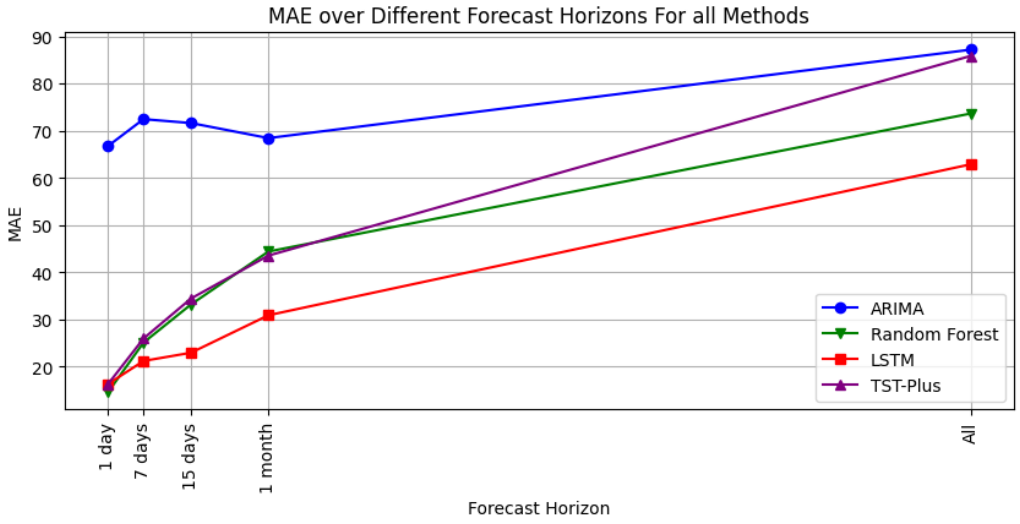


Figure 5: Evolution of MAE across different forecasting horizons for each model

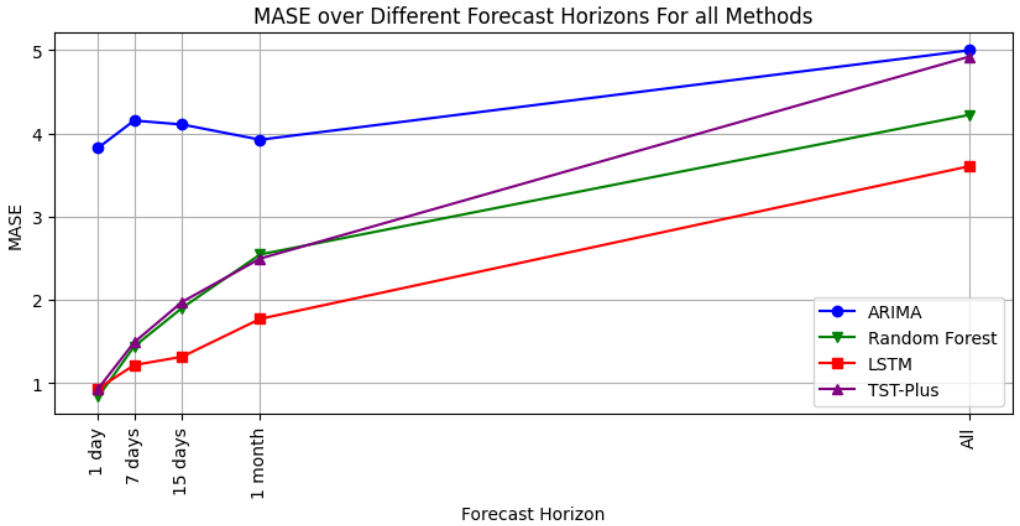


Figure 6: Evolution of MASE across different forecasting horizons for each model

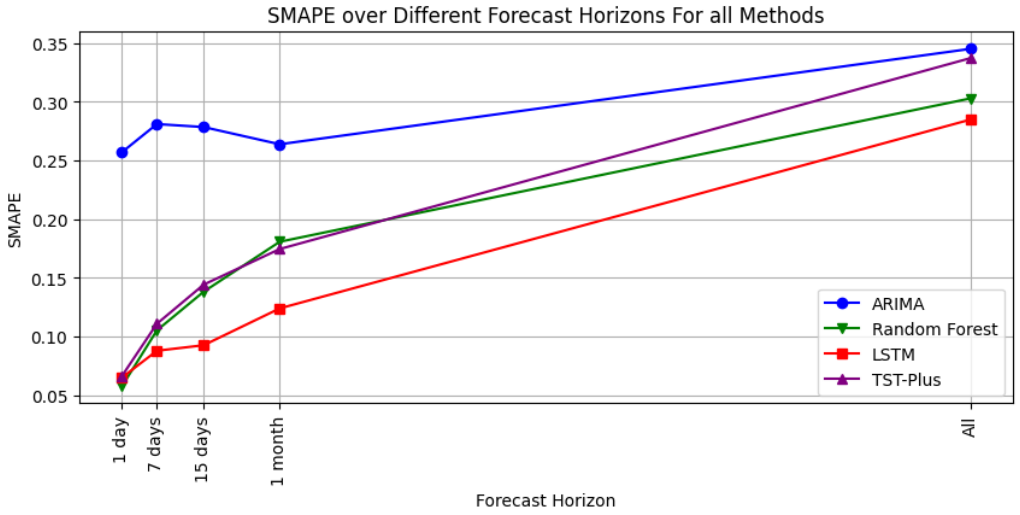


Figure 7: Evolution of SMAPE across different forecasting horizons for each model

Among all evaluated models, the LSTM consistently demonstrates superior robustness for long-term forecasting under a recursive prediction scheme. Nevertheless, absolute performance metrics remain within acceptable ranges only up to approximately 30 days ahead. According to commonly used interpretative guidelines, SMAPE values below 10% are considered highly accurate, while values up to 20% are regarded as moderately accurate [3].

Figure 8 presents the LSTM forecast over the first 30 days. Although the error metrics suggest that predictions may remain usable beyond 15 days, we would conservatively limit the practical forecasting horizon to two weeks, beyond which uncertainty becomes increasingly pronounced.

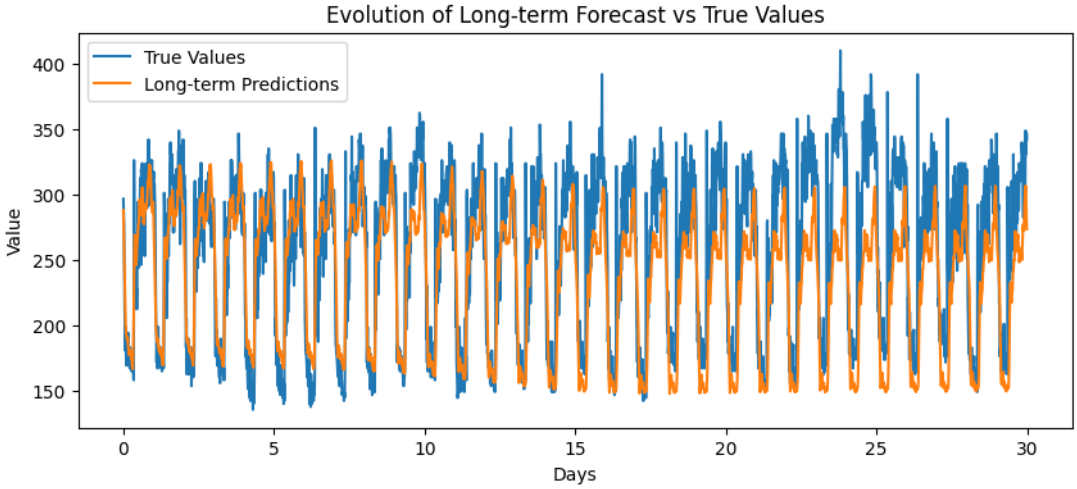


Figure 8: Long-term LSTM forecasts compared to ground truth

# Conclusion

This work presented a comparative study of four forecasting approaches drawn from three major model families: classical statistical and machine learning models (ARIMA and Random Forest), recurrent neural networks (LSTM), and transformer-based architectures.

All models were trained to perform one-day-ahead forecasting (96 time steps) using a one-week historical input window. Their performance was evaluated across multiple forecasting horizons, including one-step-ahead, one-day-ahead, and long-term recursive forecasting.

From a theoretical perspective, transformer-based models are expected to deliver superior performance due to their ability to capture complex temporal dependencies. This expectation was confirmed for the one-step-ahead forecasting task, where the transformer achieved the lowest error across all evaluated metrics. However, performance degraded more rapidly for longer forecasting horizons.

For one-day-ahead forecasting, all models except ARIMA exhibited comparable performance, suggesting that short-term daily consumption patterns can be effectively captured by a variety of modeling approaches. In contrast, long-term recursive forecasting revealed clearer differences between models, with the LSTM consistently demonstrating greater robustness to error accumulation.

Although quantitative metrics indicate that LSTM-based forecasts may remain usable for horizons of up to 30 days, a conservative operational limit of approximately 15 days is recommended, beyond which uncertainty and accumulated error become substantial.

Several avenues for future work emerge from this study. One promising direction is the use of longer forecasting windows, where models predict multiple consecutive days simultaneously rather than relying on recursive one-day-ahead forecasting. Such an approach could reduce error propagation and potentially improve long-horizon forecasting performance.

Another important direction concerns feature engineering. Incorporating additional explanatory variables, such as calendar-based features (e.g., day of the week or seasonality indicators), could provide the models with richer contextual information and lead to more accurate forecasts.

Finally, due to computational constraints, the Transformer model was trained with a limited number of parameters and training epochs. Future experiments using larger model capacities and longer training schedules may further improve Transformer performance and allow for a more comprehensive comparison between deep learning architectures.

# Bibliographie

# References

[1] Jan G De Gooijer and Rob J Hyndman. 25 years of time series forecasting. *Int. J. Forecast.*, 22(3):443–473, January 2006.

[2] Yannik Hahn, Tristan Langer, Richard Meyes, and Tobias Meisen. Time series dataset survey for forecasting with deep learning. *Forecasting*, 5(1):315–335, March 2023.

[3] Spyros Makridakis and Michèle Hibon. The M3-Competition: results, conclusions and implications. *Int. J. Forecast.*, 16(4):451–476, October 2000.

[4] Robin M Schmidt. Recurrent neural networks (RNNs): A gentle introduction and overview. 2019.

[5] Robert H Shumway and David S Stoffer. ARIMA models. In *Springer Texts in Statistics*, Springer Texts in Statistics, pages 75–163. Springer International Publishing, Cham, 2017.

[6] Artur Trindade. ElectricityLoadDiagrams20112014. UCI Machine Learning Repository, 2015. DOI: https://doi.org/10.24432/C58C86.

[7] Mohammad Valipour. Long-term runoff study using SARIMA and ARIMA models in the united states. *Meteorol. Appl.*, 22(3):592–598, July 2015.

[8] Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N Gomez, Lukasz Kaiser, and Illia Polosukhin. Attention is all you need. 2017.

[9] Muhammad Waqas and Usa Wannasingha Humphries. A critical review of RNN and LSTM variants in hydrological time series predictions. *MethodsX*, 13(102946):102946, December 2024.

[10] George Zerveas, Srideepika Jayaraman, Dhaval Patel, Anuradha Bhamidipaty, and Carsten Eickhoff. A transformer-based framework for multivariate time series representation learning. In *Proceedings of the 27th ACM SIGKDD Conference on Knowledge Discovery & Data Mining*, New York, NY, USA, August 2021. ACM.